Requirements Coverage Report

Group 44

This report maps Appendix B and the Group 44 SRS to the implementation. A tick marks a completed requirement. All 28 model tests and seven console tests passed. CLI checks also covered the SRS search examples and the Design Document's event search and update example, including reopening saved data in a new process.

1 Appendix B user stories

Story	Status
US1	✓
US2	✓
US3	✓
US4	✓
US5	✓
US6	✓
US7	✓
US8	✓
US9	✓
US10	✓
US11	✓

2 SRS functional requirements

Requirement	Status	Implementation
FR-CRE-01	✓	`RecordBook.create` and `RecordFactory.create`.
FR-CRE-06	✓	`CommandParser` rejects duplicate assignments. The factory validates before `RecordBook` advances the ID.
FR-CRE-02	✓	`Note` validates and stores the text field.
FR-CRE-03	✓	`Task` stores description and deadline.
FR-CRE-04	✓	`Event` stores its three fields and checks alarm order.
FR-CRE-05	✓	`Contact` stores all three fields as text.
FR-MOD-01	✓	`PimController.update` passes the selected ID to `RecordBook.update`.
FR-MOD-02	✓	`RecordFactory.updated` preserves type, ID and unselected fields.
FR-MOD-03	✓	`RecordBook.update` commits the validated replacement.
FR-SEA-01	✓	`TypeCriterion.matches` compares the type.
FR-SEA-02	✓	`TextCriterion.matches` uses case-sensitive containment.
FR-SEA-03	✓	`TimeCriterion.matches` compares typed times.
FR-SEA-04	✓	`AndCriterion.matches` short-circuits on false.
FR-SEA-05	✓	`OrCriterion.matches` short-circuits on true.
FR-SEA-06	✓	`NotCriterion.matches` inverts its operand.
FR-SEA-07	✓	`CriterionParser.parse` and `RecordBook.search`.
FR-SEA-08	✓	`CriterionLexer` and `CriterionParser` reject invalid input.

Requirement	Status	Implementation
FR-DIS-01	✓	`PimController.print` and `ConsoleView.showRecord`.
FR-DIS-02	✓	`RecordBook.getAll` and `ConsoleView.showRecords`.
FR-DEL-01	✓	`RecordBook.delete` removes the requested ID.
FR-DEL-02	✓	`RecordBook.delete` reports an unknown ID.
FR-FIL-01	✓	`PimController.save` confirms overwrite and `PimFileStorage.save` writes the snapshot.
FR-FIL-02	✓	`PimController.load` confirms discard and installs a valid snapshot.
FR-FIL-03	✓	`PimFileStorage.load` validates the whole file before returning.
FR-FIL-04	✓	`RecordSnapshot`, `PimFileStorage` and `RecordBook.replaceState`.
FR-ERR-01	✓	`CommandParser`, model validation and the `PimController` error loop.
FR-ERR-02	✓	`ConsoleView.showHelp` and `showSearchHelp`.

3 SRS non-functional requirements

Requirement	Status	Evidence
NFR-01	✓	Console tests verify error reporting and subsequent commands after invalid input and file failures.
NFR-02	✓	`RecordTests.invalidUpdate` and `StorageTests.roundTrip` check rejection, Unicode and mobile values. The CLI check displayed all four types.
NFR-03	✓	Console tests and CLI checks cover confirmations, error reasons and both help screens.
NFR-04	✓	The benchmark results below meet the 2,000 ms response-time target.
NFR-05	✓	`StorageTests.roundTrip` and `supportedLayouts` check mixed and empty collections. A fresh CLI process loaded a saved file.
NFR-06	✓	All 28 model tests ran without manual terminal input. Model code is under `model` and its subpackages.

NFR-04 reference measurement

The NFR-04 reference environment is an Apple M4 computer with 16 GB RAM, running macOS 26.6.2 and OpenJDK 24.0.2.

The test dataset contains 250 records of each type, with sequential IDs. Creation starts with 999 records and adds one; the other commands start with 1,000. The script resets the dataset before each run. It runs each command five times to warm up, then records the longest time from 30 runs. Timing covers command parsing, execution and UTF-8 output to a null stream. It excludes dataset setup, input waiting, file I/O and terminal rendering.

Operation	Starting PIRs	Maximum (ms)
Create one note	999	0.039
Update task description	1,000	0.086
Delete one contact	1,000	0.038
Display one event	1,000	0.059
Display all records	1,000	2.852
Search by event type, text and start time	1,000	0.766

Run the following commands from the PIM source root. The benchmark script defines the dataset and commands. The Model Line Coverage report contains the separate coverage results.

```
./scripts/test.sh
java --class-path build/classes scripts/PerformanceCheck.java
```